

Guia do Supabase — ProSeller

O Supabase é onde os dados moram. Este guia diz o que dá para fazer com tranquilidade, o que exige cuidado e o que não tem volta.

Projeto: `xxoiqfraeolsqsmsheue` · <https://supabase.com/dashboard>

A regra de ouro

Ler é seguro. Escrever, não.

Consultar dados não estraga nada — pode explorar à vontade. Mas um comando de escrita mal escrito apaga informação de verdade, e **não existe “desfazer”**.

Antes de qualquer escrita, pergunte: “se isso apagar a coluna errada, eu consigo recuperar?” Se a resposta não for um sim claro, **não execute — peça ao Claude Code**, que faz backup e ensaia antes.

As três coisas dentro do Supabase

1. Tabelas — onde os dados ficam

83 tabelas. As que importam no dia a dia:

Tabela	O que guarda
<code>cliente</code>	Cadastro dos clientes (989)
<code>cliente_endereço</code> , <code>cliente_contato</code>	Endereço e contato, separados do cadastro
<code>pedido_venda</code>	Pedidos (644)
<code>pedido_venda_produtos</code>	Os itens de cada pedido
<code>nota_fiscal_item</code>	Os itens que realmente saíram na nota
<code>vendedor_comissão</code>	Comissões calculadas
<code>frete_logistica</code>	Entregas e rastreo
<code>produto</code> , <code>produtos_listas_precos</code>	Produtos e o preço em cada lista
<code>auditoria</code>	Quem fez o quê
<code>sentinela_alerta</code>	O que a verificação diária encontrou
<code>cliente_historico_alteracoes</code>	Todo campo alterado em cliente — foi daqui que recuperamos dados perdidos

Duas armadilhas de nome:

- Algumas tabelas têm **acento e maiúscula** (`cliente_endereço` , `vendedor_comissão` , `Condicao_De_Pagamento`). Nas consultas precisam de aspas duplas: `select * from "cliente_endereço"` .
- **Nada é apagado de verdade.** As tabelas têm uma coluna `deleted_at` : quando algo é “excluído”, ela recebe a data e o registro some das telas, mas continua lá. Por isso quase toda consulta precisa de `where deleted_at is null` — **sem isso você conta registros excluídos junto e o número sai errado.**

2. Funções — a lógica que mora no banco

145 funções. São elas que salvam cliente, calculam comissão, criam pedido.

É o lugar mais perigoso do sistema. Todos os incidentes caros vieram daqui — inclusive o que apagou campos de 94 clientes e ficou quatro meses sem ninguém ver.

Regra: **nunca altere uma função sem backup e sem ensaio.** As versões anteriores ficam em `docs/plans/backups/` , com data no nome.

3. Edge Functions — o que conversa com o mundo

65 funções que ficam entre a tela e o banco, e falam com o Tiny (ERP), com a Receita e com o rastreo das transportadoras.

Sobem por comando manual:

```
npx supabase functions deploy NOME --project-ref xxoiqfraeolsqmsheue
```

| Consultando sem risco

No painel do Supabase: **SQL Editor**. Alguns exemplos úteis:

```
-- Quantos clientes ativos existem?  
select count(*) from cliente where deleted_at is null;  
  
-- Pedidos de hoje  
select numero_pedido, status, valor_total  
from pedido_venda  
where deleted_at is null and data_venda = current_date;  
  
-- O que a verificação diária encontrou e ainda não foi resolvido  
select regra, detalhe from sentinela_alerta where resolvido_em is null;  
  
-- Quem mexeu em comissão nos últimos 7 dias  
select ocorrido_em, usuario_nome, descricao  
from auditoria
```

```
where entidade = 'Comissão' and ocorrido_em > now() - interval '7 days'
order by ocorrido_em desc;
```

Repare no `where deleted_at is null` — ele aparece quase sempre, pelo motivo explicado acima.

Se precisar mesmo escrever

O jeito seguro, que funciona para qualquer alteração:

```
BEGIN;
-- sua alteração aqui
update ...;
-- confira o resultado
select ...;
ROLLBACK;
```

`BEGIN` abre um ensaio. `ROLLBACK` desfaz tudo. **Você vê o resultado real sobre os dados reais, e nada é gravado.**

Só depois de ver o resultado certo é que se troca `ROLLBACK` por `COMMIT`.

Foi assim que toda alteração desta etapa foi feita, sem nenhum acidente.

O que nunca fazer

Não faça	Por quê
<code>delete from</code> qualquer tabela	Nada aqui é apagado de verdade — o certo é marcar <code>deleted_at</code>
<code>update</code> sem <code>where</code>	Altera todas as linhas da tabela de uma vez
Alterar função sem backup	É a origem de todos os incidentes caros
Mexer em migration já aplicada	Crie uma nova; alterar as antigas quebra a reconstrução
Compartilhar a chave <code>service_role</code>	Ela ignora todas as permissões do sistema

As chaves e para que servem

No painel: **Settings** → **API**

Chave	Serve para	Cuidado
<code>anon</code>	O sistema no navegador. Respeita as permissões.	Pode aparecer no código
<code>service_role</code>	Rotinas internas. Ignora todas as permissões.	Nunca no navegador, nunca compartilhada

Chave	Serve para	Cuidado
Token de acesso	Comandos de administração pela linha de comando	Guardado no <code>.envrc</code> , fora do código

Outras configurações ficam em **Edge Functions** → **Secrets** — chave da API do Tiny, do Resend (e-mail), destinatários do alerta.

Para trocar quem recebe o e-mail da Sentinela: `SENTINELA_EMAIL_DESTINO`, vários e-mails separados por vírgula. Não precisa mexer em código.

As rotinas automáticas

Rodam dentro do próprio banco, sem depender de ninguém:

Rotina	Quando
<code>sentinela-diaria</code>	6h — confere as 15 regras
<code>sentinela-email-diaria</code>	6h15 — avisa se encontrou algo
<code>ssw-sweep-hourly</code>	de hora em hora — atualiza o rastreo

Para conferir se estão ativas:

```
select jobname, schedule, active from cron.job;
```

Testar sem tocar em produção

```
npm run test:db
```

Isso monta **uma cópia descartável do banco** na sua máquina e roda as verificações. É onde experimentar à vontade: errar ali não custa nada.

Quando alguma coisa parecer estranha

1. **Rode `npm run drift`** — ele compara o que está no ar com o código. Se alguém alterou algo direto no banco, aparece aqui.
2. **Olhe a tela Auditoria** — quem mexeu no quê, e quando.
3. **Olhe a tela Sentinela** — o que a verificação encontrou.
4. **Se envolver dinheiro** (comissão, pagamento, nota fiscal): pare e peça ajuda antes de mexer.